UNIVERSITÄT BERN

3D Geometry Processing
Exercise 07: Delaunay Triangulation

Submission deadline: 28.04.2026, 13:00

Late submissions are not accepted

Submission rules

You should submit a single `.zip` compressed file named `Ex`n`-GroupMemberNames.zip` where n is the number of the current exercise sheet. *Please order the names alphabetically by last name to make life easier for us :)* The file should contain:

- In programming exercises: all code files you changed, and **only** the files you changed (headers and source).
- A single file (`.txt`, `.pdf` or `.md`) with the following contents:
  - Names of all team members,
  - Any difficulties encountered and how you resolved them,
  - Optionally any comments about the exercise.
  - Your solutions to the theoretical exercises (if any)
  - (Only in case you used any AI assistants:) List for which purposes/problems you used them, and your judgement of where this was helpful, useless, or misleading.
- (Only in case you used any AI assistants:) Include all your chat transcripts as separate files in an archive `AI-transcripts.zip` inside the main `.zip`.

1 Theory Exercise (1 pt)

In Delaunay Triangulation we need to perform the In-Circle test to check the validity of a given edge. Specifically, an edge shared by two adjacent triangles is illegal and must be flipped if the opposite vertex lies strictly inside the other triangle's circumcircle.

For four coplanar points $A = (A_x, A_y)$, $B = (B_x, B_y)$, $C = (C_x, C_y)$, and $D = (D_x, D_y)$ (Figure 1), the `InCircle(A, B, C, D)` function tests this by evaluating the sign of the following 4×4 determinant M:

$$M = \det \begin{bmatrix} A_x & A_y & A_x^2 + A_y^2 & 1 \\ B_x & B_y & B_x^2 + B_y^2 & 1 \\ C_x & C_y & C_x^2 + C_y^2 & 1 \\ D_x & D_y & D_x^2 + D_y^2 & 1 \end{bmatrix}$$

To optimize computation, this 4×4 determinant can be simplified to a 3×3 determinant. Write out the final equivalent 3×3 matrix and

Hint: Look at the fourth column of the matrix M. How can you leverage a column of constant values to simplify the calculation? Think about the allowable row operations that preserve the value of the determinant.

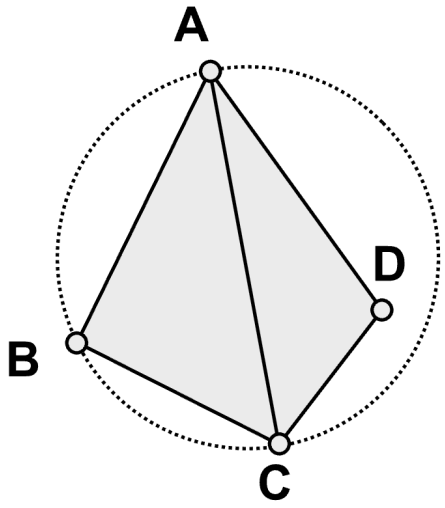


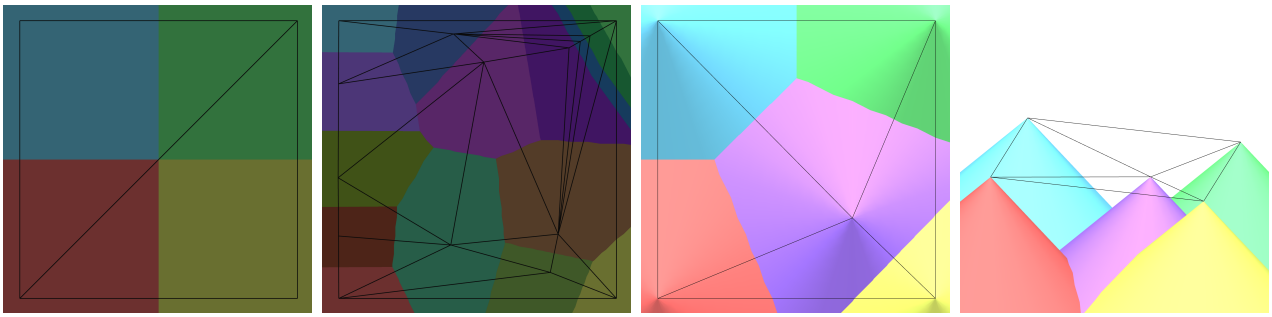
Figure 1: *In-Circle test configuration*

2 Coding Exercise (2 pts)

The goal of this exercise is to implement the Delaunay Triangulation algorithm which maximizes the minimum angle of all the angles of the triangles in the triangulation. After correct implementation, the demo will help you to build a connection between the Voronoi diagram and the triangulation in an interactive way.

- By clicking the button `Create Initial Mesh` in the plugin toolbox, the demo starts with an initial triangle mesh containing two triangles and four vertices. The corresponding Voronoi cells are visualized as projections of cones on the base plane in different colors. The Voronoi edges are the projections of the cone intersections on the base plane (See Figure 2). In case the view is changed, you can restore the perspective by pressing the `Set 2D View` button.
- In the demo, you need to incrementally add points to the triangle mesh. The picking mode should be activated by clicking on the arrow icon in the OpenFlipper toolbox. Then you can left click with your mouse *inside* the existing mesh to add a point. If the newly added point is on a mesh edge, it will perform an edge split; otherwise it will do a face split. The Voronoi diagram will be updated simultaneously.
- First, complete the implementation of `is_delaunay(...)`. It will serve as a basic operator for the next function.
- Then, add your code to the `insert_point(...)` function. The first part of the function inserts the new vertex (either on an edge or inside a triangle) and is already implemented. You thus need to implement the algorithm that checks if the new triangles meet the Delaunay condition and flips edges if not. Recall that multiple edges might have to be flipped at each insertion. Overall, your task is to make sure the Delaunay property is valid over the whole mesh.

  NOTE: Make sure you use the `mesh.is_flip_ok(...)` function in order to make sure an edge can be flipped before you try to flip it.



(a) *Initial* **(b)** *After inserting points* **(c)** *Shaded cones* **(d)** *Cones from the side*

Figure 2: *Meshes and corresponding Voronoi diagrams rendered using cones*